

Supplementary Material

S1 Prompt Templates

This appendix reproduces the exact prompt templates used in each treatment. Placeholders in angle brackets are substituted at generation time: `<query>` is the function under repair, `<context>` is the retrieved context (a Stack Overflow discussion for SOSecure and POsecure, or a CVE pattern for the CVEfixes variation), `<type_signatures>` is the 1-hop type context, and `<generated_code>` / `<build_errors>` are used only during self-healing. The type-signature block is included only when the extractor returns a non-empty result. All patch-generation prompts share the system message: *“You are a helpful assistant that generates secure JavaScript/TypeScript code. Please provide all generated code in one code block.”*

All four patch-generation prompts (No RAG, SOSecure, and both POsecure variants, as well as the CVEfixes variation) end with the same block of eight strict compilation and framework rules. To avoid repetition, we list these rules once in Listing 1 and refer to them as the STRICT RULES block in the remaining templates.

Listing 1: The STRICT RULES block, shared by all patch-generation prompts.

```
STRICT COMPILATION & FRAMEWORK RULES (CRITICAL):
1. NO NEW IMPORTS: You are modifying an isolated function block. DO NOT
  invoke external libraries (like DOMPurify, Joi, xss, etc.) or RxJS
  operators (of, throwError) unless they are ALREADY defined in the
  [LIBRARY SIGNATURES].
2. CLASS METHOD PRESERVATION: If the provided code is a class method
  (e.g., methodName(...) { ... }), return it EXACTLY as a class method.
  DO NOT prepend the function keyword or convert it to an arrow
  function property.
3. TYPESCRIPT STRICT ERRORS: In catch (error) blocks, assume error is of
  type unknown. You MUST typecast it (e.g., if (error instanceof
  Error) { ... }) before accessing properties like .message.
4. NO MODERN ES2022 ERROR CAUSES: DO NOT use the { cause: error } object
  when instantiating Errors (e.g., do not write new Error(msg,
  { cause: err })). Strictly use the standard new Error(msg).
5. STRICT TYPE PRESERVATION: Do not change the original nullability of
  variables or return types. Do not reassign variables declared with
  const.
6. ANGULAR SANITIZER AWARENESS: If you use Angular's
  this.sanitizer.sanitize(), it strictly requires TWO arguments
  (SecurityContext and the value). If context is unavailable, use
  standard JS regex/type-checking to prevent XSS.
7. ENVIRONMENT ISOLATION: Do not mix browser modules with Node.js
  modules. If the code uses Angular injections (like this.router), DO
  NOT use Node.js native modules like crypto.createHash().
8. MUST MATCH THE TYPE CONTEXT: When calling external libraries or local
  modules, strictly use the arguments and types defined in the provided
  Type Context.
```

S1.1 No RAG (baseline)

The baseline includes no retrieved context, but still provides the 1-hop type signatures and the STRICT RULES block.

```
Given the following code:

<query>

Below are the EXACT type signatures and contracts for the libraries and
local modules imported in this file. STRICTLY ADHERE to these
definitions to prevent compilation errors. Do not hallucinate parameters
or methods that do not exist in these signatures:
---
<type_signatures>
---
**Instructions:**
- Review the code for potential security flaws.
- If security issues are found, modify the code minimally to fix the
  issue while maintaining its original functionality and logic.
- If no security issues are found, output "No security issues found".
```

```
- KEEP THE EXACT SAME FUNCTION SIGNATURE (input arguments, return type
  and name).
- RETURN ONLY THE CODE. No markdown, no explanation.

[ STRICT RULES block, see Listing 1 ]
```

S1.2 SOSecure

The type-signature block precedes the vulnerability question; the retrieved Stack Overflow discussion follows it.

```
Given the following code:

<query>

Below are the EXACT type signatures and contracts for the libraries and
local modules imported in this file. STRICTLY ADHERE to these
definitions to prevent compilation errors. Do not hallucinate parameters
or methods that do not exist in these signatures:
---
<type_signatures>
---
Does this code have any security vulnerabilities? Below is a related
StackOverflow additional context that may be helpful:
---
<context>
---
**Instructions:**
- Review the code for any security flaws.
- If the code has security issues, modify the code to follow best
  security practices while ensuring the original functionality and logic
  are maintained.
- If no security issues are found, output "No security issues found".
- Ensure that the fixes do not alter the original intent of the code.
- KEEP THE EXACT SAME FUNCTION SIGNATURE (input arguments, return type
  and name).
- RETURN ONLY THE CODE. No markdown, no explanation.
- It is imperative that the new code should not have any CWE or CVE
  security errors.

[ STRICT RULES block, see Listing 1 ]
```

S1.3 POsecure (Extractive and Abstractive)

Both POsecure variants use this template, differing only in the `<context>` passed in (extractively compressed vs. abstractively summarized). Here the retrieved context precedes the type-signature block.

```
Given the following code:

<query>

Does this code have any security vulnerabilities?
Below is a related StackOverflow additional context that may be helpful:
---
<context>
---
Below are the EXACT type signatures and contracts for the libraries and
local modules imported in this file. STRICTLY ADHERE to these
definitions to prevent compilation errors. Do not hallucinate parameters
or methods that do not exist in these signatures:
---
<type_signatures>
---
**Instructions:**
- Review the code for any security flaws.
- If the code has security issues, modify the code to follow best
  security practices while ensuring the original functionality and logic
  are maintained.
- If no security issues are found, output "No security issues found".
- Ensure that the fixes do not alter the original intent of the code.
- KEEP THE EXACT SAME FUNCTION SIGNATURE (input arguments, return type
  and name).
- MUST MATCH THE TYPE CONTEXT: When calling external libraries or local
  modules, strictly use the arguments and types defined in the provided
  Type Context.
- RETURN ONLY THE CODE. No markdown, no explanation.
- It is imperative that the new code should not have any CWE or CVE
  security errors.
```

[STRICT RULES block, see Listing 1]

S1.4 CVEfixes Variation

Identical to the POSecure template, except that the retrieved context is introduced with a CVE-oriented header instead of the Stack Overflow one:

```
Below are related real-world vulnerability patterns (CVEs) and their
secure fixes that may be helpful as a reference for your correction:
---
<context>
---
```

S1.5 Self-Healing

When a generated patch fails to compile, this prompt is issued with the previously generated code and the compiler errors. It carries no retrieved context and no type-signature block.

```
You previously generated the following TypeScript code:

---typescript
<generated_code>
---
```

However, when compiling, it resulted in the following TypeScript build errors:

```
---
<build_errors>
---

**Instructions:**
- Fix the compilation errors in the code above.
- DO NOT hallucinate new imports or properties. Use only what is
  available in the scope.
- RETURN ONLY THE CORRECTED CODE. No markdown, no explanation.
- KEEP THE EXACT SAME FUNCTION SIGNATURE.
```

S2 End-to-End Example

To illustrate how the evaluated techniques operate on a complete repository, Figure S1 traces a single function of the OWASP Juice Shop backend through our implementation of SOSecure, from the vulnerable input to the patch returned by the model. It shows the retrieved Stack Overflow context, the 1-hop type signatures provided to the model, the resulting prompt, and the generated patch. We use SOSecure as the illustrative case; the remaining techniques follow the same overall flow, differing only in the retrieved context and how it is processed (prompt compression in the POSecure variants and a CVE-derived source in the CVEfixes variation).

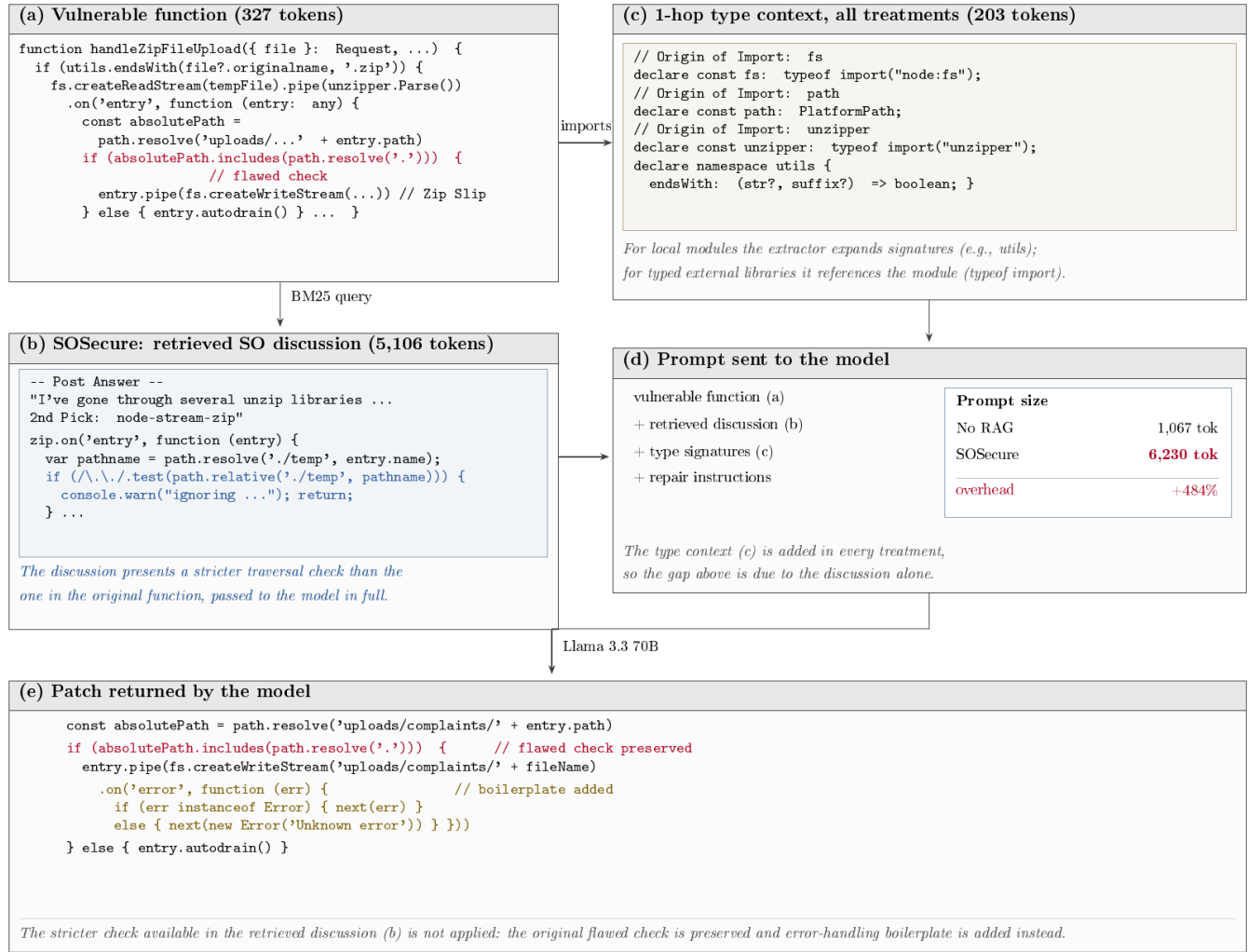


Figure S1: End-to-end application of our SOSecure implementation to a vulnerable function of the OWASP Juice Shop backend. The retrieved Stack Overflow discussion presents a stricter traversal check than the one in the original function, but the generated patch preserves the original flawed check and adds error-handling boilerplate instead. Including the full retrieved discussion raises the prompt from 1,067 to 6,230 input tokens. The other techniques follow the same flow, differing in the retrieved context and its processing.